

Reading `TaylorScan.v`, `TaylorReal.v` and `TaylorLink.v`

Zimmermann's hard-to-round search in Rocq, for a beginner

Three files in `code/APaul/rocq/` state the search of `doc/htr.md` and prove that it misses no hard-to-round case, assuming the evaluation of `exp` and its Taylor bound:

- `TaylorScan.v`: the search over the integers, and that it returns every j where the integer polynomial is close to a multiple of M ;
- `TaylorReal.v`: the step from `exp` to that integer polynomial, on the real numbers;
- `TaylorLink.v`: the two put together, in the final theorem `scan_exp`.

This note explains the idea, then the files. The three files are in the annexes. None has an **Admitted**.

1 The problem

A double x is **hard to round** when $\exp x$ is extremely close to a point where rounding changes its mind. Measured in a unit v (half the distance between two doubles near $\exp x$), the rounding points are the integers, so x is hard to round at level m when $\exp(x)/v$ is within 2^{-m} of an integer. We want every such x in a given range, without evaluating $\exp$ at each of the billions of doubles.

2 Zimmermann's idea in four steps

1. **Cut the range** into subranges of n consecutive doubles $x_0, x_0 + u, \dots, x_0 + (n-1)u$, where u is the distance between two doubles. We number them $j = 0, \dots, n-1$.
2. **Replace $\exp$ by a polynomial in j** . On a subrange, $\exp(x_0 + ju)/v$ is very close to a polynomial of small degree in j (its Taylor polynomial). Multiply by a large power of 2, $M = 2^{64\ell}$, and round the coefficients: we get a polynomial with **integer** coefficients,

$$P(j) = A_0 + A_1j + \dots + A_{k-1}j^{k-1}.$$

Only the part of $P(j)$ modulo M matters, because only the distance to an integer matters.

3. **Test each j** . $x_0 + ju$ is a **candidate** when $P(j)$ is within a margin E of a multiple of M . E is chosen to cover everything we neglected: the target 2^{-m} , the Taylor remainder, and the rounding of the coefficients.
4. **Compute $P(0), P(1), P(2), \dots$ with additions only**, by the table of differences (next section). This is what makes the search fast.

The test in one formula. “ b is within E of a multiple of M ” is checked as

$$(b + E) \bmod M \leq 2E.$$

Example with $M = 100$ and $E = 5$: $b = 197$ is within 3 of 200, and $(197 + 5) \bmod 100 = 2 \leq 10$: a candidate. $b = 150$ gives $(150 + 5) \bmod 100 = 55 > 10$: not a candidate. Adding E shifts the interval $[-E, E]$ around a multiple of M to $[0, 2E]$, which a single comparison can test.

3 The table of differences

The **difference** of a sequence f is the sequence

$$\Delta f(j) = f(j+1) - f(j).$$

Applying it again gives the higher differences, each one the difference of the previous one:

$$\Delta^0 f = f, \quad \Delta^{i+1} f(j) = \Delta^i f(j+1) - \Delta^i f(j).$$

For a polynomial of degree d , each difference lowers the degree by one. So the d -th difference $\Delta^d f(j)$ does not depend on j : it is $d!$ times the leading coefficient; and the $(d + 1)$ -th difference is 0 for every j . The **table** at j is the list $f(j), \Delta f(j), \Delta^2 f(j), \dots$

Example with $f(j) = j^2$:

- $\Delta f(j) = (j + 1)^2 - j^2 = 2j + 1$;
- $\Delta^2 f(j) = (2(j + 1) + 1) - (2j + 1) = 2$, which does not depend on j ($2!$ times the leading coefficient 1);
- $\Delta^3 f(j) = 2 - 2 = 0$.

So the table at $j = 0$ is $[0, 1, 2]$. **One step** adds each row to the one below it, and gives the table at $j + 1$:

j	$f(j)$	$\Delta f(j)$	$\Delta^2 f(j)$
0	0	1	2
1	$0 + 1 = 1$	$1 + 2 = 3$	2
2	$1 + 3 = 4$	$3 + 2 = 5$	2
3	$4 + 5 = 9$	$5 + 2 = 7$	2

The first column gives 0, 1, 4, 9: the squares, with additions only. This is **Shift.v**, already proved:

- **dtab d f j** is the table of f at j , with $d + 1$ rows;
- **tstep** is one step;
- **is_poly d f** says that f is a polynomial of degree at most d , that is $f(j) = l_0 C(j, 0) + \dots + l_d C(j, d)$ for some coefficients l_i (the binomial basis: the coefficients need not be divided); **is_polyP** says this holds exactly when $\Delta^{d+1} f$ is zero everywhere;
- **tstepE**: if f is a polynomial of degree at most d (**is_poly d f**), one step turns the table at j into the table at $j + 1$.

4 The file, line by line

Notations used. **int** is the type of integers of the library **mathcomp**. $x *+ n$ is x added n times, that is nx . $(a \% M)\%Z$ is $a \bmod M$, between 0 and $M - 1$. $|x|$ is the absolute value. **iota 0 n** is the list $0, 1, \dots, n - 1$, and **[seq j <- s | p j]** keeps the elements j of s for which **p j** holds. **j \in s** means j is in the list s . $(j < n)\%N$ is a comparison of natural numbers.

The data. One subrange is described by four values:

Variables (**A : seq int**) (**M E : int**) (**n : nat**).

the coefficients $A_0, \dots, A_{k-1}$ as a list, the modulus M , the margin E , and the number n of doubles.

The polynomial.

Definition **P : nat -> int := hpoly A.**

hpoly (from **Shift.v**) is the polynomial with coefficients **A** in Horner form: $A_0 + j(A_1 + j(A_2 + \dots))$. Its degree is at most the length of **A**:

Definition **dg := size A.**

Lemma **is_poly_P : is_poly dg P.**

The test, the formula of section 2:

Definition **hit (b : int) : bool := ((b + E) \% M)\%Z <= E *+ 2.**

The walk, step 6 of the note. It keeps the current table **t** and the current index **j**, tests the first row, then takes one step; **c** counts the doubles left:

```

Fixpoint walk (j : nat) (t : seq int) (c : nat) : seq nat :=
  if c is c'.+1 then
    let rest := walk j.+1 (tstep t) c' in
    if hit (nth 0 t 0) then j :: rest else rest
  else [::].

```

`nth 0 t 0` is the first row, the current value $P(j)$. The result is the list of candidates.

The search starts the walk from the table of P at 0:

Definition `scan : seq nat := walk 0 (dtab dg P 0) n`.

5 What is proved

The walk computes the right thing.

Lemma `scanE : scan = [seq j <- iota 0 n | hit (P j)]`.

The walk returns exactly the $j < n$ whose $P(j)$ passes the test: the fast computation with additions gives the same answer as evaluating P at every j . The proof (`walkE`) goes by induction on the number of doubles left; at each step `tstepE` says the table after one step is the table at the next index, whose first row is $P(j+1)$.

The test catches every b close to a multiple of M .

Lemma `hitP (b w : int) : E *+ 2 < M -> `|b - M * w| <= E -> hit b`.

If $|b - Mw| \leq E$, then $b - Mw + E$ is between 0 and $2E$, which is less than M , so it is $(b + E) \bmod M$. The condition $2E < M$ makes sure the interval $[0, 2E]$ does not wrap around.

The main theorem.

Theorem `scan_complete j (w : int) :`
`E *+ 2 < M -> (j < n)%N -> `|P j - M * w| <= E -> j \in scan`.

If $P(j)$ is within E of a multiple of M , then j is a candidate. It is `scanE` and `hitP` put together. The file has no `Admitted`.

6 From exp to the scan: TaylorReal.v

`scan_complete` talks about P , not about `exp`. `TaylorReal.v` makes the step between them: **if $x_0 + ju$ is hard to round, then $P(j)$ is within E of a multiple of M** . It holds under the three assumptions of the note. With $a_i = \exp(x_0)u^i/(i!v)$ the exact coefficients and ρ a bound on the Taylor remainder:

- (H_A) each A_i is within 1 of M times the fractional part of a_i ; this is the evaluation of `exp`, done once per subrange;
- (H_T) $\exp(x_0 + ju)/v$ is within ρ of $a_0 + a_1j + \dots + a_{k-1}j^{k-1}$;
- (H_E) $E \geq M(2^{-m} + \rho) + (1 + n + \dots + n^{k-1})$.

The file is plain Rocq on real numbers (the `Stdlib` library, no `mathcomp`). In it, `exp` does not appear: the value $\exp(x_0 + ju)/v$ is a real y , and 2^{-m} a real ε . Two definitions:

- `sumR k f` is $f(0) + \dots + f(k-1)$;
- `Pz A k j` is the integer $A_0 + A_1j + \dots + A_{k-1}j^{k-1}$.

The lemma:

```

Lemma real_lemma (k n j : nat) (M E : Z) (a : nat -> R) (A : nat -> Z)
  (rho eps y : R) :
  (0 < M)%Z -> (j <= n)%nat ->
  (forall i, (i < k)%nat -> Rabs (IZR (A i) - IZR M * frac_part (a i)) < 1) ->
  Rabs (y - sumR k (fun i => a i * INR j ^ i)) <= rho ->

```

```

IZR M * (eps + rho) + sumR k (fun i => INR n ^ i) <= IZR E ->
(exists z : Z, Rabs (y - IZR z) < eps) ->
exists w : Z, (Z.abs (Pz A k j - M * w) <= E)%Z.

```

IZR and INR turn an integer and a natural number into a real; `frac_part r` is r minus its integer part. The three middle hypotheses are (H_A), (H_T) and (H_E); the last one says y is within ε of an integer z .

The proof in four lines. Write each a_i as its integer part plus its fractional part.

1. $Ma_i j^i$ and $M \text{frac}(a_i) j^i$ differ by M times an integer, because j^i is an integer. So $M \times \sum a_i j^i$ is $\sum M \text{frac}(a_i) j^i$ plus MK for an integer K .
2. Replacing each $M \text{frac}(a_i)$ by A_i costs less than $j^i \leq n^i$, by (H_A): the total error is at most $1 + n + \dots + n^{k-1}$.
3. Replacing $\sum a_i j^i$ by y costs at most $M\rho$, by (H_T); and y is within ε of z , so My is within $M\varepsilon$ of Mz .
4. Take $w = z - K$: $P(j) - Mw$ is the sum of the three errors, at most E by (H_E).

The file proves it with five small lemmas on `sumR`: two sums can be added, multiplied by a constant, compared term by term, bounded by the triangle inequality, and `Pz read` in the reals is the real sum (`IZR_Pz`).

7 The final theorem: `TaylorLink.v`

`TaylorScan.v` is written with the `mathcomp` library, whose integers are the type `int` and whose polynomial is `hpoly A`; `TaylorReal.v` uses Rocq's `Z` and `Pz`. `TaylorLink.v` connects them:

- `hpoly_Pz`: `hpoly A j` and `Pz` compute the same integer (`Z_of_int` turns a `mathcomp int` into a `Z`);
- `scan_completeZ`: `scan_complete`, restated with `Z` and `Pz`;
- `scan_exp`: `real_lemma` followed by `scan_completeZ`.

```

Theorem scan_exp (A : seq int) (M E : int) (n j : nat) (a : nat -> R)
  (rho eps y : R) :
  Z.lt 0 (Z_of_int M) -> Z.lt (Z.mul 2 (Z_of_int E)) (Z_of_int M) ->
  (j < n)%N ->
  (H_A) -> (H_T) -> (H_E) ->
  (exists z : Z, Rlt (Rabs (Rminus y (IZR z))) eps) ->
  j \in scan A M E n.

```

(written here with (H_A), (H_T), (H_E) for the three hypotheses; the file spells them out). In words: **if $\exp(x_0 + ju)/v$ is within 2^{-m} of an integer, then j is a candidate**. The search misses no hard-to-round case. The file writes the real and `Z` operations as functions (`Rle`, `Rplus`, `Z.add`, ...) because `mathcomp` takes over the infix notations $\leq$, $+$, ... for its own types.

Print `Assumptions scan_exp` lists only the two axioms Rocq's real numbers are built on.

8 What is assumed

- (H_A), the evaluation of `exp`: for `htr2_fix.c`, it is the bound of its Tang evaluator at 448 bits, not yet proved.
- (H_T), the Taylor bound for `exp` with its remainder: a theorem of analysis, not yet proved here.
- Two details separate the files from the C program `htr2_fix.c`:
 - the C builds the first table in place from $P(0), \dots, P(k-1)$ (steps 3 and 4 of the note), where `TaylorScan.v` takes `dtab` directly; and its table has one more row, which is 0;
 - the C reduces every number modulo M after each addition, where the file keeps exact integers and reduces only in the test; the two give the same test, because reduction modulo M commutes with addition.

A Annex: TaylorScan.v

```
(** * Zimmermann's search (doc/htr.md): the scan of one subrange

On a subrange of [n] consecutive doubles [x0 + j u], the note replaces
[exp(x0 + j u) / v] by an integer polynomial in [j],
  [P j = A_0 + A_1 j + ... + A_(k-1) j^(k-1)],
whose coefficients are the [A_i] of the note, and walks it with the
table of differences of [Shift.v]. A [j] is a candidate when
[(P j + E) mod M <= 2 E], with [M = beta^l].

This file states the algorithm over the integers and proves the
part that does not depend on [exp]:
- [scanE]: the walk returns exactly the [j < n] passing the test;
- [scan_complete]: every [j] with [P j] within [E] of a multiple of
  [M] is returned.
The link to [exp] is in [TaylorReal.v] and [TaylorLink.v]. *)

From mathcomp Require Import all_ssreflect all_algebra.
From APaulRocq Require Import Shift.

Set Implicit Arguments.
Unset Strict Implicit.
Unset Printing Implicit Defensive.

Import GRing.Theory Num.Theory.
Local Open Scope ring_scope.

Section Scan.

(** The data of a subrange: the [A_i], least significant first, the
    modulus [M = beta^l], the window [E], and the number of doubles. *)
Variables (A : seq int) (M E : int) (n : nat).

(** The polynomial, in Horner form. *)
Definition P : nat -> int := hpoly A.

(** Its degree is at most [size A], so a table of [size A + 1] rows walks
    it exactly (the note uses [size A] rows: the last one here is [0]). *)
Definition dg := size A.

Lemma is_poly_P : is_poly dg P.
Proof. exact: is_poly_hpoly. Qed.

(** The test on the first row of the table: steps 5 and 6 of the note. *)
Definition hit (b : int) : bool := ((b + E) %% M)%Z <= E *+ 2.

(** The main loop, step 6: test the first row, then shift the table. *)
Fixpoint walk (j : nat) (t : seq int) (c : nat) : seq nat :=
  if c is c'.+1 then
    let rest := walk j.+1 (tstep t) c' in
    if hit (nth 0 t 0) then j :: rest else rest
  else [::].

(** The search: the table at [0], steps 3 and 4, then the walk. *)
Definition scan : seq nat := walk 0 (dtab dg P 0) n.
```

```

(** The walk returns exactly the [j] in [[i, i + c]] whose [P j] passes
    the test. *)
Lemma walkE i c : walk i (dtab dg P i) c = [seq j <- iota i c | hit (P j)].
Proof.
elim: c i => [//|c IH] i /=.
rewrite tstepE; last exact: is_poly_P.
by rewrite IH.
Qed.

Lemma scanE : scan = [seq j <- iota 0 n | hit (P j)].
Proof. exact: walkE. Qed.

(** If [b] is within [E] of a multiple [M w] of [M], and [2 E < M], then
    [b] passes the test: [(b + E) mod M] is [b - M w + E], in [[0, 2 E]]. *)
Lemma hitP (b w : int) : E * 2 < M -> `|b - M * w| <= E -> hit b.
Proof.
move=> EM; rewrite ler_norml => /andP[l r].
have lo : 0 <= b - M * w + E by rewrite -lerBlDr sub0r.
have hi : b - M * w + E < M.
  by apply: Order.POrderTheory.le_lt_trans EM; rewrite mulr2n lerD2r.
rewrite /hit.
have -> : b + E = w * M + (b - M * w + E).
  by rewrite [w * M]mulrC addrA [M * w + _]addrC subrK.
rewrite modzMDl modz_small ?lo ?hi //.
by rewrite mulr2n lerD2r.
Qed.

(** The completeness of the scan, with no reference to [exp]. *)
Theorem scan_complete j (w : int) :
  E * 2 < M -> (j < n)%N -> `|P j - M * w| <= E -> j \in scan.
Proof.
move=> EM jn Pw.
by rewrite scanE mem_filter (hitP EM Pw) /= mem_iota /= add0n.
Qed.

End Scan.

(** The step from [exp] to the hypothesis of [scan_complete] is
    [TaylorReal.real_lemma]; [TaylorLink.scan_exp] puts the two
    together. *)

```

B Annex: TaylorReal.v

```

(** * Zimmermann's search (doc/htr.md): from the reals to the scan

[TaylorScan.scan_complete] says: if the integer polynomial [P] is
within [E] of a multiple of [M] at [j], then [j] is a candidate. This
file proves the step before it, on the reals: if [y] (standing for
[exp(x0 + j u) / v]) is within [eps] (standing for [2^-m]) of an
integer, then [P j] is within [E] of a multiple of [M], under the three
hypotheses of the note:
  (H_A) each [A_i] is within 1 of [M frac(a_i)];
  (H_T) [y] is within [rho] of [sum_i a_i j^i];
  (H_E) [E >= M (eps + rho) + (1 + n + ... + n^(k-1))].

```

[exp] does not appear: it only enters through (H_A) and (H_T).

Stdlib only, no mathcomp, so that the real arithmetic is not disturbed by mathcomp's notations. *)

From Stdlib Require Import ZArith Reals Lra Lia.

Open Scope R_scope.

```
(** [sum_(i < k) f i], by recursion on [k]. *)
Fixpoint sumR (k : nat) (f : nat -> R) : R :=
  match k with 0 => 0 | S k' => sumR k' f + f k' end.
```

```
(** The integer polynomial [sum_(i < k) A_i j^i]. *)
Fixpoint Pz (A : nat -> Z) (k j : nat) : Z :=
  match k with
  | 0 => 0%Z
  | S k' => (Pz A k' j + A k' * Z.of_nat j ^ Z.of_nat k')%Z
  end.
```

```
(** Extensionality of [sumR] below [k]. *)
Lemma sumR_ext (k : nat) (f g : nat -> R) :
  (forall i, (i < k)%nat -> f i = g i) -> sumR k f = sumR k g.
Proof.
induction k as [|k IH]; intros H; simpl; [reflexivity|].
rewrite IH by (intros i Hi; apply H; lia).
rewrite H by lia; reflexivity.
Qed.
```

```
(** [sumR] is additive. *)
Lemma sumR_plus (k : nat) (f g : nat -> R) :
  sumR k (fun i => f i + g i) = sumR k f + sumR k g.
Proof. induction k as [|k IH]; simpl; [lra|rewrite IH; lra]. Qed.
```

```
(** [sumR] commutes with a scalar factor. *)
Lemma sumR_scal (k : nat) (c : R) (f : nat -> R) :
  sumR k (fun i => c * f i) = c * sumR k f.
Proof. induction k as [|k IH]; simpl; [ring|rewrite IH; ring]. Qed.
```

```
(** The triangle inequality for [sumR]. *)
Lemma Rabs_sumR_le (k : nat) (f g : nat -> R) :
  (forall i, (i < k)%nat -> Rabs (f i) <= g i) ->
  Rabs (sumR k f) <= sumR k g.
Proof.
induction k as [|k IH]; intros H; simpl.
- rewrite Rabs_R0; lra.
- apply Rle_trans with (Rabs (sumR k f) + Rabs (f k)); [apply Rabs_triangular|].
  apply Rplus_le_compat; [apply IH; intros i Hi; apply H; lia|apply H; lia].
Qed.
```

```
(** The integer polynomial, read in the reals. *)
Lemma IZR_Pz (A : nat -> Z) (k j : nat) :
  IZR (Pz A k j) = sumR k (fun i => IZR (A i) * INR j ^ i).
Proof.
induction k as [|k IH]; simpl; [reflexivity|].
rewrite plus_ISR, mult_ISR, IH, <- pow_ISR, <- INR_ISR_INZ.
```

reflexivity.

Qed.

```
Lemma real_lemma (k n j : nat) (M E : Z) (a : nat -> R) (A : nat -> Z)
  (rho eps y : R) :
  (0 < M)%Z -> (j <= n)%nat ->
  (forall i, (i < k)%nat -> Rabs (IZR (A i) - IZR M * frac_part (a i)) < 1) ->
  Rabs (y - sumR k (fun i => a i * INR j ^ i)) <= rho ->
  IZR M * (eps + rho) + sumR k (fun i => INR n ^ i) <= IZR E ->
  (exists z : Z, Rabs (y - IZR z) < eps) ->
  exists w : Z, (Z.abs (Pz A k j - M * w) <= E)%Z.
```

Proof.

intros HM Hjn HA HT HE [z Hz].

set (K := Pz (fun i => Int_part (a i)) k j).

set (S := sumR k (fun i => a i * INR j ^ i)).

set (D := sumR k (fun i =>

(IZR (A i) - IZR M * frac_part (a i)) * INR j ^ i)).

assert (HP : IZR (Pz A k j) = D + IZR M * S - IZR M * IZR K).

{ unfold D, S, K; rewrite !IZR_Pz, <- !sumR_scal.

transitivity (sumR k (fun i =>

((IZR (A i) - IZR M * frac_part (a i)) * INR j ^ i

+ IZR M * (a i * INR j ^ i))

+ (-1) * (IZR M * (IZR (Int_part (a i)) * INR j ^ i)))).

{ apply sumR_ext; intros i _; unfold frac_part; ring. }

rewrite sumR_plus, sumR_scal, sumR_plus; ring. }

assert (HD : Rabs D <= sumR k (fun i => INR n ^ i)).

{ apply Rabs_sumR_le; intros i Hi.

rewrite Rabs_mult, (Rabs_right (INR j ^ i))

by (apply Rle_ge, pow_le, pos_INR).

apply Rle_trans with (1 * INR j ^ i).

- apply Rmult_le_compat_r; [apply pow_le, pos_INR|].

apply Rlt_le, HA; exact Hi.

- rewrite Rmult_1_l; apply pow_incr; split; [apply pos_INR|].

apply le_INR; exact Hjn. }

exists (z - K)%Z.

apply le_IZR; rewrite abs_IZR.

rewrite minus_IZR, mult_IZR, minus_IZR, HP.

assert (HM' : 0 < IZR M) by (apply IZR_lt; exact HM).

replace (D + IZR M * S - IZR M * IZR K - IZR M * (IZR z - IZR K))

with (D - IZR M * (y - S) + IZR M * (y - IZR z)) by ring.

apply Rle_trans

with (Rabs D + IZR M * Rabs (y - S) + IZR M * Rabs (y - IZR z)).

{ apply Rle_trans

with (Rabs (D - IZR M * (y - S)) + Rabs (IZR M * (y - IZR z)));

[apply Rabs_triangular].

rewrite Rabs_mult, (Rabs_right (IZR M)) by lra.

apply Rplus_le_compat_r.

unfold Rminus at 1; eapply Rle_trans; [apply Rabs_triangular|].

rewrite Rabs_Ropp, Rabs_mult, (Rabs_right (IZR M)) by lra; lra. }

assert (IZR M * Rabs (y - S) <= IZR M * rho)

by (apply Rmult_le_compat_l; [lra|exact HT]).

assert (IZR M * Rabs (y - IZR z) <= IZR M * eps)

by (apply Rmult_le_compat_l; lra).

lra.

Qed.

C Annex: TaylorLink.v

```
(** * Zimmermann's search (doc/htr.md): linking the reals to the scan

[TaylorReal.real_lemma] ends on [Pz], a sum over [Z], and
[TaylorScan.scan_complete] starts on [hpoly], a Horner form over
[int]. This file proves they are the same polynomial, and restates
[scan_complete] over [Z], in the shape [real_lemma] produces. *)

From Stdlib Require Import ZArith Reals Lia.
From APaulRocq Require Import TaylorReal.
From mathcomp Require Import all_ssreflect all_algebra ssrZ zify.
From APaulRocq Require Import Shift TaylorScan.

Set Implicit Arguments.
Unset Strict Implicit.
Unset Printing Implicit Defensive.

Import GRing.Theory Num.Theory.

(** One more term of [Pz], without letting [/=] unfold the power. *)
Lemma PzE (f : nat -> Z) k j :
  Pz f k.+1 j =
    Z.add (Pz f k j) (Z.mul (f k) (Z.pow (Z.of_nat j) (Z.of_nat k))).
Proof. by []. Qed.

(** Peeling the constant term off [Pz]: the Horner step, over [Z]. *)
Lemma PzS (f : nat -> Z) k j :
  Pz f k.+1 j = Z.add (f 0) (Z.mul (Z.of_nat j) (Pz (fun i => f i.+1) k j)).
Proof.
elim: k => [|k IH]; first by rewrite !PzE /=; lia.
by rewrite PzE IH PzE Nat2Z.inj_succ Z.pow_succ_r; lia.
Qed.

(** [hpoly] and [Pz] compute the same integer. *)
Lemma hpoly_Pz (A : seq int) (j : nat) :
  Z_of_int (hpoly A j) = Pz (fun i => Z_of_int (nth 0%R A i)) (size A) j.
Proof.
elim: A => [|a A IH].
by rewrite [size _] /= PzS /= -IH; lia.
Qed.

(** [scan_complete] over [Z], with the polynomial written as [Pz]. *)
Theorem scan_completeZ (A : seq int) (M E : int) (n j : nat) (w : Z) :
  Z.lt (Z.mul 2 (Z_of_int E)) (Z_of_int M) -> (j < n)%N ->
  Z.le (Z.abs (Z.sub (Pz (fun i => Z_of_int (nth 0%R A i)) (size A) j)
    (Z.mul (Z_of_int M) w)))
    (Z_of_int E) ->
  j \in scan A M E n.
Proof.
rewrite -hpoly_Pz => EM jn Pw.
by apply: (@scan_complete A M E n j (int_of_Z w) _ jn); rewrite /P; lia.
Qed.

(** The search misses nothing. [y] stands for  $\exp(x_0 + j u) / v$ , [a i]
for the Taylor coefficient  $\exp(x_0) u^i / (i! v)$ , [eps] for  $2^{-m}$ .
```

Under the three hypotheses of the note, (H_A) on the [A_i], (H_T) the Taylor bound [rho], and (H_E) on [E], if [y] is within [eps] of an integer then [j] is a candidate. Real numbers are written with prefix functions ([Rle], [Rplus], ...), since mathcomp owns the infix ones. *)

```

Theorem scan_exp (A : seq int) (M E : int) (n j : nat) (a : nat -> R)
  (rho eps y : R) :
  Z.lt 0 (Z_of_int M) -> Z.lt (Z.mul 2 (Z_of_int E)) (Z_of_int M) ->
  (j < n)%N ->
  (forall i, Peano.lt i (size A) ->
    Rlt (Rabs (Rminus (IZR (Z_of_int (nth 0%R A i)))
      (Rmult (IZR (Z_of_int M)) (frac_part (a i))))) 1) ->
  Rle (Rabs (Rminus y (sumR (size A) (fun i => Rmult (a i) (pow (INR j) i)))))
    rho ->
  Rle (Rplus (Rmult (IZR (Z_of_int M)) (Rplus eps rho))
    (sumR (size A) (fun i => pow (INR n) i)))
    (IZR (Z_of_int E)) ->
  (exists z : Z, Rlt (Rabs (Rminus y (IZR z))) eps) ->
  j \in scan A M E n.

```

Proof.

```

move=> M0 EM jn HA HT HE Hz.
have jn' : Peano.le j n by apply/leP; exact: ltnW.
have [w Hw] := real_lemma _ _ _ _ _ _ _ _ M0 jn' HA HT HE Hz.
exact: scan_completeZ EM jn Hw.
Qed.

```